DATA STRUCTURES AND ALGORITHMS LAB - Practical Programs

Each program is complete: copy it into one file, compile/run, and it works. Part A = C++, Part B = Python.

- C++: `g++ program.cpp -o program` then `./program` (Windows: `program.exe`)
- Python: `python program.py`

PART A - C++

Program 1

Question: Create a student database containing Student ID, Name, and CGPA. Allocate memory dynamically, add student records, and search for a student using ID with Linear Search.

```C/C++
#include <iostream>
#include <string>
using namespace std;

struct Student {
    int id;
    string name;
    float cgpa;
};

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;

    Student *s = new Student[n];   // dynamic memory allocation

    for (int i = 0; i < n; i++) {
        cout << "\nStudent " << i + 1 << endl;
        cout << "ID: ";
        cin >> s[i].id;
        cout << "Name (one word): ";
        cin >> s[i].name;
```

```
        cout << "CGPA: ";
        cin >> s[i].cgpa;
    }

    int key, found = 0;
    cout << "\nEnter ID to search: ";
    cin >> key;

    // Linear Search
    for (int i = 0; i < n; i++) {
        if (s[i].id == key) {
            cout << "Student found!" << endl;
            cout << "ID: " << s[i].id << ", Name: " << s[i].name
                 << ", CGPA: " << s[i].cgpa << endl;
            found = 1;
            break;
        }
    }

    if (found == 0)
        cout << "Student not found." << endl;

    delete[] s;   // free memory
    return 0;
}
```

Note: names are entered as one word (e.g. Ravi).

Program 2

Question: Store student IDs in an array, sort them in ascending order, and implement Binary Search to find a given student ID

```
C/C++
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cout << "Enter number of student IDs: ";
    cin >> n;

    int id[100];
    cout << "Enter the IDs: ";
```

```
    for (int i = 0; i < n; i++)
        cin >> id[i];

    sort(id, id + n);   // ascending order

    cout << "Sorted IDs: ";
    for (int i = 0; i < n; i++)
        cout << id[i] << " ";
    cout << endl;

    int key;
    cout << "Enter ID to search: ";
    cin >> key;

    // Binary Search
    int low = 0, high = n - 1, pos = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (id[mid] == key) {
            pos = mid;
            break;
        } else if (id[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    if (pos != -1)
        cout << "ID found at position " << pos + 1 << endl;
    else
        cout << "ID not found." << endl;

    return 0;
}
```

Program 3

Question: Store student ID, name, and CGPA. Implement Bubble Sort or Selection Sort to arrange records by CGPA in ascending order.

```
C/C++
#include <iostream>
#include <string>
using namespace std;

struct Student {
```

```
    int id;
    string name;
    float cgpa;
};

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;

    Student s[100];
    for (int i = 0; i < n; i++) {
        cout << "\nStudent " << i + 1 << endl;
        cout << "ID: ";
        cin >> s[i].id;
        cout << "Name (one word): ";
        cin >> s[i].name;
        cout << "CGPA: ";
        cin >> s[i].cgpa;
    }

    // Bubble Sort by CGPA (ascending)
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (s[j].cgpa > s[j + 1].cgpa) {
                Student temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }

    cout << "\nRecords sorted by CGPA (ascending):" << endl;
    cout << "ID\tName\tCGPA" << endl;
    for (int i = 0; i < n; i++)
        cout << s[i].id << "\t" << s[i].name << "\t" << s[i].cgpa << endl;

    return 0;
}
```

Program 4

Question: Store student IDs in an array, sort them in ascending order, and implement Binary Search to find a given student ID.

C/C++

```cpp
#include <iostream>
using namespace std;

// Recursive Binary Search
int binarySearch(int a[], int low, int high, int key) {
    if (low > high)
        return -1;
    int mid = (low + high) / 2;
    if (a[mid] == key)
        return mid;
    else if (a[mid] < key)
        return binarySearch(a, mid + 1, high, key);
    else
        return binarySearch(a, low, mid - 1, key);
}

int main() {
    int n;
    cout << "Enter number of student IDs: ";
    cin >> n;

    int id[100];
    cout << "Enter the IDs: ";
    for (int i = 0; i < n; i++)
        cin >> id[i];

    // Bubble Sort (ascending)
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (id[j] > id[j + 1]) {
                int temp = id[j];
                id[j] = id[j + 1];
                id[j + 1] = temp;
            }
        }
    }

    cout << "Sorted IDs: ";
    for (int i = 0; i < n; i++)
        cout << id[i] << " ";
    cout << endl;

    int key;
    cout << "Enter ID to search: ";
    cin >> key;

    int pos = binarySearch(id, 0, n - 1, key);
```

```
    if (pos != -1)
        cout << "ID found at position " << pos + 1 << endl;
    else
        cout << "ID not found." << endl;

    return 0;
}
```

Program 5

Question: Implement a Stack ADT using a singly linked list with push, pop, and display operations. Handle stack underflow.

C/C++

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

Node *top = NULL;

void push(int x) {
    Node *newNode = new Node;
    newNode->data = x;
    newNode->next = top;
    top = newNode;
    cout << x << " pushed." << endl;
}

void pop() {
    if (top == NULL) {
        cout << "Stack Underflow! Stack is empty." << endl;
        return;
    }
    Node *temp = top;
    cout << temp->data << " popped." << endl;
    top = top->next;
    delete temp;
}

void display() {
    if (top == NULL) {
        cout << "Stack is empty." << endl;
```

```
        return;
    }
    cout << "Stack elements (top to bottom): ";
    Node *p = top;
    while (p != NULL) {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}

int main() {
    int choice, x;
    do {
        cout << "\n1. Push\n2. Pop\n3. Display\n4. Exit\nEnter choice: ";
        cin >> choice;
        if (choice == 1) {
            cout << "Enter value: ";
            cin >> x;
            push(x);
        } else if (choice == 2) {
            pop();
        } else if (choice == 3) {
            display();
        } else if (choice != 4) {
            cout << "Invalid choice." << endl;
        }
    } while (choice != 4);
    return 0;
}
```

Program 6

Question: Implement a stack using an array and convert a valid infix expression into postfix notation, considering operator precedence and parentheses.

C/C++

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

char st[100];   // stack using array
int top = -1;

void push(char c) { st[++top] = c; }
```

```
char pop() { return st[top--]; }

int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int main() {
    string infix, postfix = "";
    cout << "Enter infix expression (no spaces, e.g. A+B*(C-D)): ";
    cin >> infix;

    for (int i = 0; i < infix.length(); i++) {
        char c = infix[i];

        if (isalnum(c)) {                // operand
            postfix += c;
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (top != -1 && st[top] != '(')
                postfix += pop();
            pop();                       // remove '('
        } else {                         // operator
            while (top != -1 && st[top] != '(' &&
                   (prec(st[top]) > prec(c) ||
                   (prec(st[top]) == prec(c) && c != '^')))
                postfix += pop();
            push(c);
        }
    }

    while (top != -1)
        postfix += pop();

    cout << "Postfix expression: " << postfix << endl;
    return 0;
}
```

Enter the expression without spaces, e.g. `A+B*(C-D)/E`.

Program 7

Question: Implement a stack using an array and convert a valid infix expression into prefix notation.

C/C++

```
#include <iostream>
#include <string>
#include <algorithm>
#include <cctype>
using namespace std;

char st[100];   // stack using array
int top = -1;

void push(char c) { st[++top] = c; }
char pop() { return st[top--]; }

int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int main() {
    string infix, postfix = "";
    cout << "Enter infix expression (no spaces, e.g. A+B*(C-D)): ";
    cin >> infix;

    // Step 1: reverse the expression
    reverse(infix.begin(), infix.end());

    // Step 2: swap '(' and ')'
    for (int i = 0; i < infix.length(); i++) {
        if (infix[i] == '(') infix[i] = ')';
        else if (infix[i] == ')') infix[i] = '(';
    }

    // Step 3: convert to postfix
    for (int i = 0; i < infix.length(); i++) {
        char c = infix[i];

        if (isalnum(c)) {
            postfix += c;
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (top != -1 && st[top] != '(')
                postfix += pop();
            pop();
        } else {
```

```
            while (top != -1 && st[top] != '(' &&
                   (prec(st[top]) > prec(c) ||
                   (prec(st[top]) == prec(c) && c == '^')))
                postfix += pop();
            push(c);
        }
    }
    while (top != -1)
        postfix += pop();

    // Step 4: reverse the result to get prefix
    reverse(postfix.begin(), postfix.end());

    cout << "Prefix expression: " << postfix << endl;
    return 0;
}
```

Program 8

Question: Implement a Circular Queue of using an array of size 5 in C++. Write a program to perform Enqueue, Dequeue, and Display operations. Handle full and empty queue conditions.

C/C++

```
#include <iostream>
using namespace std;

#define SIZE 5

int q[SIZE];
int front = -1, rear = -1;

void enqueue(int x) {
    if ((rear + 1) % SIZE == front) {
        cout << "Queue is FULL!" << endl;
        return;
    }
    if (front == -1)
        front = 0;
    rear = (rear + 1) % SIZE;
    q[rear] = x;
    cout << x << " enqueued." << endl;
}

void dequeue() {
    if (front == -1) {
        cout << "Queue is EMPTY!" << endl;
```

```
        return;
    }
    cout << q[front] << " dequeued." << endl;
    if (front == rear)
        front = rear = -1;          // queue became empty
    else
        front = (front + 1) % SIZE;
}

void display() {
    if (front == -1) {
        cout << "Queue is EMPTY!" << endl;
        return;
    }
    cout << "Queue elements: ";
    int i = front;
    while (true) {
        cout << q[i] << " ";
        if (i == rear) break;
        i = (i + 1) % SIZE;
    }
    cout << endl;
}

int main() {
    int choice, x;
    do {
        cout << "\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\nEnter choice: ";
        cin >> choice;
        if (choice == 1) {
            cout << "Enter value: ";
            cin >> x;
            enqueue(x);
        } else if (choice == 2) {
            dequeue();
        } else if (choice == 3) {
            display();
        } else if (choice != 4) {
            cout << "Invalid choice." << endl;
        }
    } while (choice != 4);
    return 0;
}
```

PART B - Python

Program 9

Question: Create a BST by inserting integer values. Display inorder, preorder, and postorder traversals. Handle duplicate values according to a defined rule.

Python

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None


# Rule for duplicates: a duplicate value is NOT inserted (it is ignored)
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    else:
        print("Duplicate", key, "ignored")
    return root


def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)


def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)


def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
```

```
        print(root.key, end=" ")


values = list(map(int, input("Enter values separated by space: ").split()))

root = None
for v in values:
    root = insert(root, v)

print("Inorder   : ", end="")
inorder(root)
print("\nPreorder  : ", end="")
preorder(root)
print("\nPostorder : ", end="")
postorder(root)
print()
```

Program 10

Question: Implement insertion and search operations in a BST. Display whether a given key is present.

Python

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None


def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root


def search(root, key):
    if root is None:
        return False
    if root.key == key:
        return True
    if key < root.key:
```

```
        return search(root.left, key)
    return search(root.right, key)


values = list(map(int, input("Enter values to insert (space separated):
").split()))

root = None
for v in values:
    root = insert(root, v)

key = int(input("Enter key to search: "))

if search(root, key):
    print("Key", key, "is PRESENT in the BST")
else:
    print("Key", key, "is NOT present in the BST")
```

Program 11

Question: Implement deletion of a specified key from a BST, handling leaf nodes, nodes with one child, and nodes with two children.

Python

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None


def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root


def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)
```

```
def min_node(node):
    while node.left:
        node = node.left
    return node


def delete(root, key):
    if root is None:
        print("Key not found")
        return None
    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        # Case 1: leaf node
        if root.left is None and root.right is None:
            return None
        # Case 2: one child
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        # Case 3: two children -> use inorder successor
        succ = min_node(root.right)
        root.key = succ.key
        root.right = delete(root.right, succ.key)
    return root


values = list(map(int, input("Enter values to insert (space separated):
").split()))

root = None
for v in values:
    root = insert(root, v)

print("Inorder before deletion: ", end="")
inorder(root)
print()

key = int(input("Enter key to delete: "))
root = delete(root, key)

print("Inorder after deletion : ", end="")
inorder(root)
```

```
print()
```

Program 12

Question: Find the depth of a BST and display all leaf nodes.

Python

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None


def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root


# depth = number of levels (empty tree = 0)
def depth(root):
    if root is None:
        return 0
    return 1 + max(depth(root.left), depth(root.right))


def leaves(root):
    if root is None:
        return
    if root.left is None and root.right is None:
        print(root.key, end=" ")
    leaves(root.left)
    leaves(root.right)


values = list(map(int, input("Enter values to insert (space separated):
").split()))

root = None
for v in values:
    root = insert(root, v)
```

```
print("Depth of BST (levels):", depth(root))
print("Leaf nodes: ", end="")
leaves(root)
print()
```

Program 13

Question: Represent a graph of your college campus using adjacency list. Implement Kruskal's algorithm to find its Minimum Spanning Tree and display selected edges and total weight.

Python

```
# Campus graph using adjacency list: building -> [(neighbour, distance), ...]
graph = {
    "Gate":      [("Admin", 4), ("Canteen", 8)],
    "Admin":     [("Gate", 4), ("Library", 3), ("CS Dept", 6)],
    "Library":   [("Admin", 3), ("CS Dept", 2), ("Hostel", 7)],
    "CS Dept":   [("Admin", 6), ("Library", 2), ("Canteen", 5)],
    "Canteen":   [("Gate", 8), ("CS Dept", 5), ("Hostel", 4)],
    "Hostel":    [("Library", 7), ("Canteen", 4)],
}

# Step 1: collect every edge once (weight, u, v)
edges = []
for u in graph:
    for v, w in graph[u]:
        if (w, v, u) not in edges:
            edges.append((w, u, v))

# Step 2: sort edges by weight
edges.sort()

# Union-Find to detect cycles
parent = {node: node for node in graph}

def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

# Step 3: Kruskal's algorithm
mst = []
total = 0
for w, u, v in edges:
    root_u = find(u)
    root_v = find(v)
```

```
    if root_u != root_v:          # no cycle
        parent[root_u] = root_v
        mst.append((u, v, w))
        total += w

print("Edges in Minimum Spanning Tree:")
for u, v, w in mst:
    print(u, "-", v, ":", w)
print("Total weight of MST =", total)
```

Program 14

Question: Represent a graph of your college campus using adjacency matrix. Implement Prim's algorithm to find the Minimum Spanning Tree and calculate its total weight.

Python

```
# Campus graph using adjacency matrix (0 = no direct path)
names = ["Gate", "Admin", "Library", "CS Dept", "Canteen", "Hostel"]

matrix = [
    #  G  A  L  C  Ca H
    [0, 4, 0, 0, 8, 0],   # Gate
    [4, 0, 3, 6, 0, 0],   # Admin
    [0, 3, 0, 2, 0, 7],   # Library
    [0, 6, 2, 0, 5, 0],   # CS Dept
    [8, 0, 0, 5, 0, 4],   # Canteen
    [0, 0, 7, 0, 4, 0],   # Hostel
]

n = len(names)
INF = 999999

selected = [False] * n      # vertices already in MST
key = [INF] * n             # smallest edge weight to reach each vertex
parent = [-1] * n

key[0] = 0                  # start from vertex 0
total = 0

for _ in range(n):
    # pick the unselected vertex with the smallest key
    u = -1
    for i in range(n):
        if not selected[i] and (u == -1 or key[i] < key[u]):
            u = i
```

```
    selected[u] = True
    total += key[u]

    # update keys of neighbours
    for v in range(n):
        if matrix[u][v] != 0 and not selected[v] and matrix[u][v] < key[v]:
            key[v] = matrix[u][v]
            parent[v] = u

print("Edges in Minimum Spanning Tree (Prim's):")
for v in range(1, n):
    print(names[parent[v]], "-", names[v], ":", matrix[parent[v]][v])
print("Total weight of MST =", total)
```

Program 15

Question: Design and implement the Heap Sort algorithm to efficiently sort an array of integers in ascending order. The implementation should be optimized for time and space complexity and should clearly demonstrate the working principles of heap data structures (min-heap or max-heap as applicable)

Python

```
# Heap Sort using a MAX-HEAP (in-place, time O(n log n), extra space O(1))

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


def heap_sort(arr):
    n = len(arr)

    # Step 1: build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
```

```
    # Step 2: move the largest element to the end, then fix the heap
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)


arr = list(map(int, input("Enter numbers separated by space: ").split()))
print("Original array:", arr)
heap_sort(arr)
print("Sorted array  :", arr)
```

Program 16

Question: You are given a list of online orders, each with an estimated delivery time in minutes. Write a program to sort these orders using the Merge Sort algorithm so the delivery system can prioritize quicker deliveries first.

Python

```
def merge_sort(orders):
    if len(orders) <= 1:
        return orders

    mid = len(orders) // 2
    left = merge_sort(orders[:mid])
    right = merge_sort(orders[mid:])

    # merge two sorted halves (sort by delivery time)
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i][1] <= right[j][1]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result


n = int(input("Enter number of orders: "))
orders = []
for i in range(n):
```

```
    t = int(input("Delivery time (minutes) for Order " + str(i + 1) + ": "))
    orders.append(("Order " + str(i + 1), t))

sorted_orders = merge_sort(orders)

print("\nOrders sorted by delivery time (quickest first):")
for name, t in sorted_orders:
    print(name, "->", t, "minutes")
```

Program 17

Question: You run a shipping company and need to load a truck with parcels of different weights and profits. The truck has a limited weight capacity. Write a program to choose parcels (even partially) to maximize profit using the Fractional Knapsack strategy.

Python

```
# Fractional Knapsack - greedy method (highest profit/weight first)

n = int(input("Enter number of parcels: "))
parcels = []
for i in range(n):
    w = float(input("Weight of parcel " + str(i + 1) + ": "))
    p = float(input("Profit of parcel " + str(i + 1) + ": "))
    parcels.append((i + 1, w, p))

capacity = float(input("Enter truck capacity: "))

# sort by profit/weight ratio (highest first)
parcels.sort(key=lambda x: x[2] / x[1], reverse=True)

total_profit = 0
remaining = capacity

print("\nParcels loaded:")
for num, w, p in parcels:
    if remaining == 0:
        break
    if w <= remaining:                  # take whole parcel
        print("Parcel", num, ": full  (weight", w, ", profit", p, ")")
        total_profit += p
        remaining -= w
    else:                               # take a fraction
        fraction = remaining / w
        print("Parcel", num, ": partial", round(fraction * 100, 2), "% (profit",
round(p * fraction, 2), ")")
```

```
        total_profit += p * fraction
        remaining = 0

print("\nMaximum profit =", round(total_profit, 2))
```

Program 18

Question: A text string text of length n. A pattern string pattern of length m. Find all starting indices i in the text such that the substring text[i:i+m] is exactly equal to the pattern using the Naive String Matching Algorithm approach. Constraints: 0 ≤ m ≤ n

Python

```
# Naive String Matching Algorithm

text = input("Enter text   : ")
pattern = input("Enter pattern: ")

n = len(text)
m = len(pattern)

found = []

for i in range(n - m + 1):
    if text[i:i + m] == pattern:      # compare substring with pattern
        found.append(i)

if len(found) == 0:
    print("Pattern not found")
else:
    print("Pattern found at indices:", found)
```

Program 19

Question: Represent a small maze as a two-dimensional grid containing free cells and obstacles. Use Breadth-First Search (BFS) to find the shortest path from a start cell to a goal cell. Display the path or report that no path exists.

Python

```
from collections import deque

# 0 = free cell, 1 = obstacle
maze = [
    [0, 0, 1, 0, 0],
```

```
    [1, 0, 1, 0, 1],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
]

start = (0, 0)
goal = (4, 4)

# BFS
queue = deque([start])
parent = {start: None}

while queue:
    r, c = queue.popleft()
    if (r, c) == goal:
        break
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 5 and 0 <= nc < 5 and maze[nr][nc] == 0 and (nr, nc) not in
parent:
            parent[(nr, nc)] = (r, c)
            queue.append((nr, nc))

if goal not in parent:
    print("No path exists")
else:
    path = []
    cell = goal
    while cell is not None:
        path.append(cell)
        cell = parent[cell]
    path.reverse()

    print("Shortest path length:", len(path) - 1)
    print("Path:", path)
```

Program 20

Question: Implement A* search for a small grid maze using Manhattan distance as the heuristic. Display the path from start to goal.

Python

```
import heapq

# 0 = free cell, 1 = obstacle
```

```
maze = [
    [0, 0, 1, 0, 0],
    [1, 0, 1, 0, 1],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
]

start = (0, 0)
goal = (4, 4)

def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])   # Manhattan distance

# A* search
open_list = [(heuristic(start, goal), 0, start)]   # (f, g, cell)
g_cost = {start: 0}
parent = {start: None}

while open_list:
    f, g, cell = heapq.heappop(open_list)
    if cell == goal:
        break
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = cell[0] + dr, cell[1] + dc
        if 0 <= nr < 5 and 0 <= nc < 5 and maze[nr][nc] == 0:
            new_g = g + 1
            if (nr, nc) not in g_cost or new_g < g_cost[(nr, nc)]:
                g_cost[(nr, nc)] = new_g
                parent[(nr, nc)] = cell
                f_new = new_g + heuristic((nr, nc), goal)
                heapq.heappush(open_list, (f_new, new_g, (nr, nc)))

if goal not in parent:
    print("No path exists")
else:
    path = []
    cell = goal
    while cell is not None:
        path.append(cell)
        cell = parent[cell]
    path.reverse()

    print("Path length:", len(path) - 1)
    print("Path:", path)
```